

Project: FleetIQ – Real-Time Logistics & Fleet Optimization Platform

1. Introduction

This document outlines the Low-Level Design (LLD) for **FleetIQ**, a real-time logistics and fleet optimization platform designed for third-party logistics (3PL) providers. The system manages vehicle assignments, route optimization, delivery tracking, and predictive maintenance to improve operational efficiency.

Supports backend development using **Java (Spring Boot)** and **.NET (ASP.NET Core)**.

2. Module Overview

- 2.1 Fleet & Vehicle Management Module
- 2.2 Route Optimization & Assignment Module
- 2.3 Delivery Tracking & Proof of Delivery Module
- 2.4 Maintenance Scheduling & Alerts Module
- 2.5 Logistics Analytics & SLA Reporting Module

3. Architecture Overview

3.1 Architectural Style

- **Frontend:** Angular/React for dispatch and tracking dashboards.
- **Backend:** REST APIs for fleet operations and analytics.
- **Database:** PostgreSQL/MySQL for structured logistics data.

3.2 Component Interaction

- Dispatch team uses frontend to assign vehicles and monitor deliveries.
- Backend handles route calculations, vehicle status, and SLA metrics.

4. Module-Wise Design

4.1 Fleet & Vehicle Management Module

4.1.1 Features

- Add/update vehicle details, assign drivers, track availability.
- Monitor fuel usage and vehicle health.

4.1.2 Entities

- **Vehicle**
 - VehicleID
 - Type
 - Capacity
 - DriverID
 - Status

4.2 Route Optimization & Assignment Module

4.2.1 Features

- Optimize delivery routes based on traffic, distance, and delivery windows.
- Assign vehicles to routes dynamically.

4.2.2 Entities

- **RoutePlan**
 - RouteID
 - Origin
 - Destination
 - Distance
 - EstimatedTime

4.3 Delivery Tracking & Proof of Delivery Module

4.3.1 Features

- Real-time GPS tracking, delivery status updates.
- Capture digital proof of delivery (signature/photo).

4.3.2 Entities

- **Delivery**
 - DeliveryID
 - VehicleID
 - RouteID
 - Status
 - POD (Proof of Delivery)

4.4 Maintenance Scheduling & Alerts Module

4.4.1 Features

- Schedule preventive maintenance, track service history.

- Alert for overdue maintenance and breakdowns.

4.4.2 Entities

- **MaintenanceRecord**
 - MaintenanceID
 - VehicleID
 - Date
 - Type
 - Status

4.5 Logistics Analytics & SLA Reporting Module

4.5.1 Features

- Analyze delivery efficiency, vehicle utilization, SLA adherence.
- Generate reports for clients and internal teams.

4.5.2 Entities

- **LogisticsReport**
 - ReportID
 - Metrics
 - SLACompliance
 - GeneratedDate

5. Deployment Strategy

5.1 Local Deployment

- Developer machines with simulated GPS data.

5.2 Staging and Production Environments

- Cloud deployment with real-time data ingestion and map APIs.

6. Database Design

6.1 Tables and Relationships

- Vehicle → RoutePlan → Delivery → MaintenanceRecord → LogisticsReport

7. User Interface Design

7.1 Wireframes

- Fleet Dashboard: Vehicle status, route assignments.
- Delivery Tracker: Map view with live vehicle positions.
- Maintenance Calendar: Upcoming service schedules.

8. Non-Functional Requirements

8.1 Performance

- Support 1,500 concurrent dispatch operations.

8.2 Usability

- Mobile app for drivers with offline sync.
- Intuitive dashboards for dispatchers.

8.3 Security

- Secure APIs for GPS and POD uploads.
- Role-based access for drivers, dispatchers, and admins.

8.4 Scalability

- Multi-hub support with centralized fleet control.

9. Assumptions and Constraints

9.1 Assumptions

- Vehicles are GPS-enabled, and drivers use mobile apps.
- Clients require SLA-based delivery reports.

9.2 Constraints

- Initial rollout for intra-city deliveries only.